Nhận dạng chữ số viết tay MNIST: So sánh Naive Bayes, hồi quy softmax, MLP, Random Forest và CNN, kèm giao diện demo

Đồ án 1 – Môn Nhận dạng
Khoa Toán – Tin học, Trường ĐH Khoa học Tự nhiên, ĐHQG-HCM

Tháng 11/2026

Nhóm 10: Nguyễn Diệu Lê, Nguyễn Hồng Linh

Tóm tắt

Nhóm so sánh bốn bộ phân lớp thuộc bốn họ khác nhau: Multinomial Naive Bayes (MNB), hồi quy softmax huấn luyện bằng mini-batch gradient descent có momentum (LR), mạng nơ-ron nhiều lớp huấn luyện bằng lan truyền ngược (MLP), cả ba tự cài đặt bằng NumPy, và Random Forest (RF) của scikit-learn. Phần nâng cao gồm một CNN nhỏ, một tổ hợp soft voting và một giao diện demo cho người dùng vẽ chữ số rồi xem các mô hình dự đoán. Mọi mô hình được huấn luyện trên cùng một cách chia MNIST (50.000/10.000/10.000), chọn siêu tham số chỉ trên validation, và đối chiếu với scikit-learn hoặc PyTorch. Phân tích khám phá dữ liệu (EDA) dẫn tới bốn giả thuyết, cả bốn đều được kết quả ủng hộ: (**H1**) LR chính xác hơn và cho xác suất đáng tin hơn MNB; (**H2**) RF và MLP vượt LR; (**H3**) lỗi của mô hình tuyến tính dồn vào 5 cặp lớp có tâm gần nhau, và mô hình phi tuyến giảm chính các lỗi đó; (**H4**) vùng ảnh mà mọi mô hình dựa vào trùng với vùng có Fisher score cao. Accuracy trên test: MNB 83,56%, LR 92,38%, RF 96,68%, MLP 98,36%; CNN 98,82% và tổ hợp 5 CNN + RF 99,15%.

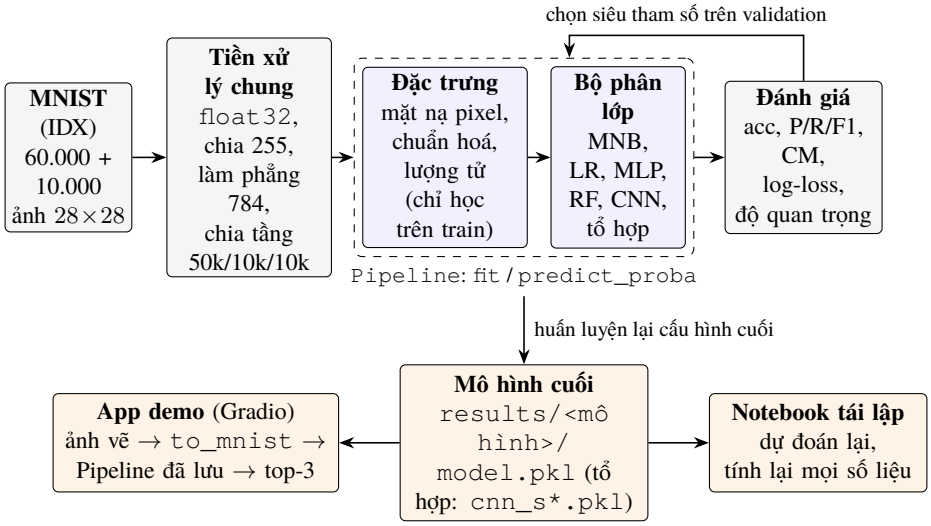
Từ khoá: MNIST, Naive Bayes, hồi quy softmax, lan truyền ngược, Random Forest, CNN, độ quan trọng hoán vị, giao diện demo.

1 Giới thiệu

MNIST [1] gồm 70.000 ảnh xám 28×28 của chữ số viết tay, là bài toán chuẩn để so sánh các bộ phân lớp. Trên MNIST, accuracy của các phương pháp hiện đại đã bão hoà gần 100%, nên nhóm quan tâm không phải "mô hình nào cao nhất" mà là: *các họ mô hình khác nhau ở đâu, và vì sao*. Nhóm so sánh theo bốn trục (độ chính xác, thời gian huấn luyện, thời gian dự đoán, khả năng giải thích) và độ tin cậy của xác suất dự đoán.

Đóng góp của báo cáo:

- MNB, LR, MLP và CNN tự cài bằng NumPy, kể cả lan truyền ngược của MLP và CNN; RF (nhánh ensemble) dùng `RandomForestClassifier` của scikit-learn. Với bốn mô hình tự cài, scikit-learn và PyTorch dùng để đối chiếu: accuracy trên test chênh với bản thư viện cùng cấu hình không quá 0,13 điểm phần trăm ở MNB, MLP và CNN; với LR, bản tự cài đạt đúng accuracy test của nghiệm hội tụ (lbfgs).
- Một chuỗi lập luận nối EDA với thiết kế mô hình: mỗi quyết định tiền xử lý xuất phát từ một phát hiện EDA, và bốn giả thuyết **H1**–**H4** được chốt *trước* khi chạy thí nghiệm.
- Quy trình đánh giá thống nhất: chọn cấu hình chỉ trên validation, test dùng đúng một lần; cùng một định nghĩa thời gian; cùng một cách đo độ quan trọng (hoán vị khối 4×4) cho mọi mô hình.
- Một giao diện demo (phụ lục B) dùng lại đúng các Pipeline đã lưu, kèm bước chuẩn hoá ảnh vẽ về dạng MNIST.



Hình 1: Kiến trúc hệ thống. Mọi mô hình là một `Pipeline` (đặc trưng + bộ phân lớp) có chung giao diện, nên cùng một quy trình đánh giá, cùng notebook tái lập và cùng app demo dùng được cho mọi mô hình. Bước đặc trưng nằm trong Pipeline nên chỉ học trên train và đi theo mô hình khi lưu.

2 Dữ liệu và phân tích khám phá

2.1 Dữ liệu và tiền xử lý chung

Dùng bản MNIST gốc định dạng IDX (60.000 ảnh train, 10.000 ảnh test). Tiền xử lý chung cho mọi mô hình chỉ gồm các bước xử lý thô: ép kiểu `uint8` sang `float32` *trước* khi chia cho 255 (tránh tràn số), làm phẳng $28 \times 28 \to 784$, và tách 10.000 ảnh validation từ tập train theo tầng (stratified, seed 42), giữ tỉ lệ lớp. Kết quả là 50.000/10.000/10.000 ảnh cho train/val/test. Mọi biến đổi có tham số (mặt nạ pixel, chuẩn hoá, lượng tử hoá) chỉ học từ tập train, để val và test không rò rỉ vào mô hình. EDA làm trên 50.000 ảnh train.

2.2 Các phát hiện EDA

(a) **Pixel gần nhị phân và nhiều vị trí trống.** 80,86% giá trị pixel bằng 0; trong các pixel khác 0, 69,2% có giá trị ≥ 128. 72 vị trí không có mực ở bất kỳ ảnh nào (*dead pixel*, phương sai 0) và 116 vị trí có mực ở dưới 0,1% số ảnh, đều nằm ở viền (Hình 2a). 99,76% lượng mực nằm trong khung 24×24 ở giữa.

(b) **Tương quan cục bộ.** Hai pixel kề nhau có hệ số tương quan 0,708; tương quan giảm còn khoảng 0,16 ở khoảng cách 3 và khoảng 0 ở khoảng cách 5. Tương quan vẫn còn khi đã biết lớp (0,64–0,86 tuỳ lớp), tức giả định độc lập có điều kiện của Naive Bayes bị vi phạm rõ.

(c) **Các lớp khác nhau cả về tâm lẫn phân tán.** Tổng phương sai trong lớp dao động 22,4 (lớp 1) đến 50,8 (lớp 2) (Hình 2b), nên ranh giới tối ưu không tuyến tính. Khoảng cách giữa tâm hai lớp từ 2,48 đến 7,56; năm cặp gần nhất là 4–9, 7–9, 5–8, 3–5, 3–8, và các cặp này chỉ khác nhau ở vài chục pixel.

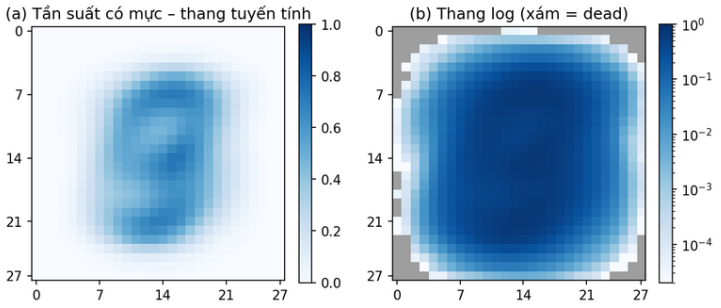
(d) **Thông tin phân biệt tập trung ở giữa ảnh.** Fisher score của pixel j [2, 3] là tỉ số phương sai giữa lớp trên phương sai trong lớp:

$$F_j = \frac{\sum_c n_c(\mu_{cj} - \mu_j)^2}{\sum_c n_c \sigma_{cj}^2},$$

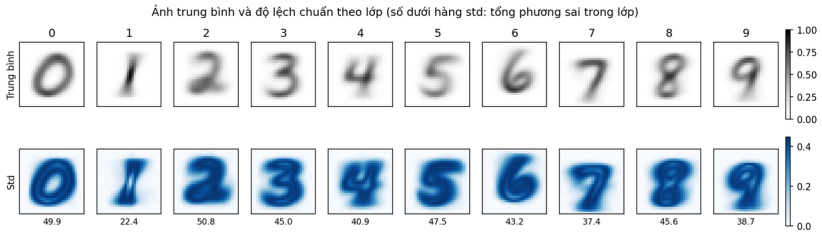
với n_c là số ảnh lớp c, μ_{cj} và σ_{cj}^2 là trung bình và phương sai của pixel j trong lớp c, μ_j là trung bình chung (pixel có mực ở dưới 0,1% số ảnh bị bỏ qua vì ước lượng không tin cậy). F_j lớn khi tâm các lớp cách xa nhau ở pixel j so với độ phân tán trong từng lớp. Fisher score dồn vào vùng giữa: khoảng 129 pixel giữ một nửa tổng Fisher (Hình 2c).

2.3 Từ phát hiện tới quyết định thiết kế

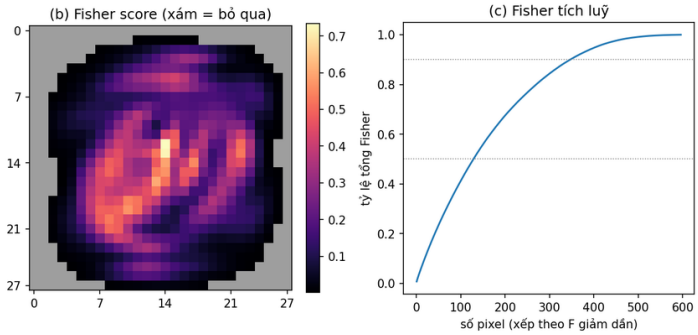
Bảng 1 tóm tắt cách mỗi phát hiện được chuyển thành quyết định cho từng mô hình. Hai lựa chọn đáng chú ý: (i) với LR, MLP và CNN, dùng *chuẩn hoá vô hướng* (một trung bình, một độ lệch chuẩn cho mọi pixel) thay cho chuẩn hoá từng pixel, vì chuẩn hoá từng pixel chia cho 0 ở dead pixel và khuếch đại các pixel hiếm (trên val,



(a) Tần suất có mực tại mỗi vị trí, thang tuyến tính và thang log (xám: dead pixel).



(b) Ảnh trung bình và độ lệch chuẩn theo lớp; số dưới hàng std là tổng phương sai trong lớp.



(c) Fisher score theo pixel (xám: bỏ qua) và tỉ lệ Fisher tích luỹ.

Hình 2: Các phát hiện EDA chính trên 50.000 ảnh train.

max $|z|$ lên tới 1314 so với 2,7); (ii) chọn Multinomial thay vì Gaussian Naive Bayes, vì pixel là "lượng mực" không âm, gần nhị phân, không có dạng chuẩn.

Bảng 1: Từ phát hiện EDA tới quyết định thiết kế.

Phát hiện	Quyết định
(a) Pixel gần nhị phân, không âm	MNB trên cường độ $[0, 1]$; RF nhị phân hoá "có mực" (thí nghiệm chọn 2 bin)
(a) 72 dead, 116 pixel hiếm ở viền	Mặt nạ pixel là siêu tham số (`none`/`dead`/`rare`/crop); làm trơn Laplace $\alpha > 0$ cho MNB; chuẩn hoá vô hướng cho LR, MLP, CNN
(b) Tương quan cục bộ	L2 cho LR; L2 hoặc dropout cho MLP; độ quan trọng đo theo *khối* 4×4 thay vì từng pixel; tích chập 3×3 cho CNN
(c) Ranh giới không tuyến tính	So mô hình phi tuyến (RF, MLP) với LR (**H2**)
(c) 5 cặp tâm gần	Theo dõi riêng 5 cặp trong ma trận nhầm lẫn (**H3**)
(d) Fisher dồn ở giữa	So bản đồ độ quan trọng của mọi mô hình với Fisher (**H4**)

2.4 Giả thuyết

Chốt trước khi thí nghiệm, kiểm chứng trên validation:

H1 LR chính xác hơn và cho xác suất đáng tin hơn MNB. Hai mô hình có cùng dạng hàm $\text{softmax}(Wx + b)$; MNB cộng bằng chứng từ các pixel tương quan như thể độc lập (phát hiện b), nên dự đoán *quá tự tin*.

H2 RF và MLP vượt LR về accuracy và F1 macro (phát hiện c).

H3 (a) Ít nhất 22,2% số lỗi của mô hình tuyến tính (gấp đôi mức rải đều $5/45$) nằm ở 5 cặp tâm gần, và ít nhất 3/5 cặp nằm trong top 5 cặp nhầm nhiều nhất; (b) so với LR, RF và MLP giảm lỗi ở 5 cặp này nhiều hơn mức giảm tổng lỗi.

H4 Độ quan trọng theo khối 4×4 của mỗi mô hình có tương quan hạng Spearman [4] $\rho \geq 0{,}7$ với tổng Fisher của khối (ρ là hệ số tương quan Pearson tính trên thứ hạng, nên chỉ đòi hỏi hai đại lượng cùng tăng giảm, không đòi hỏi quan hệ tuyến tính).

3 Phương pháp

Ký hiệu: $x \in \mathbb{R}^d$ là ảnh sau tiền xử lý, $y \in \{1, \ldots, K\}$, $K = 10$, N là số ảnh train. Mọi mô hình có cùng giao diện `fit`/`predict`/`predict_proba` và được ghép với bước tiền xử lý trong một pipeline, nên bước tiền xử lý luôn học trên train. Mỗi mô hình có một hình pipeline (Hình 3–8), đọc từ trái sang phải: ảnh đã qua tiền xử lý chung đi qua *tầng EDA*, gồm các phát hiện ở mục 2 (ô vàng, nhãn (a)–(d)) phục vụ mô hình đó và quyết định mà mỗi phát hiện dẫn tới; các nhánh hợp lại thành bước đặc trưng rồi bộ phân lớp. Softmax được tính qua log-sum-exp (trừ giá trị lớn nhất trước khi lấy mũ) để không tràn số.

3.1 Multinomial Naive Bayes

Coi cường độ pixel $x_j \in [0, 1]$ là "lượng mực" rơi vào vị trí j. Với tổng cường độ $N_{cj} = \sum_{i:y_i=c} x_{ij}$, làm trơn Laplace $\alpha > 0$:

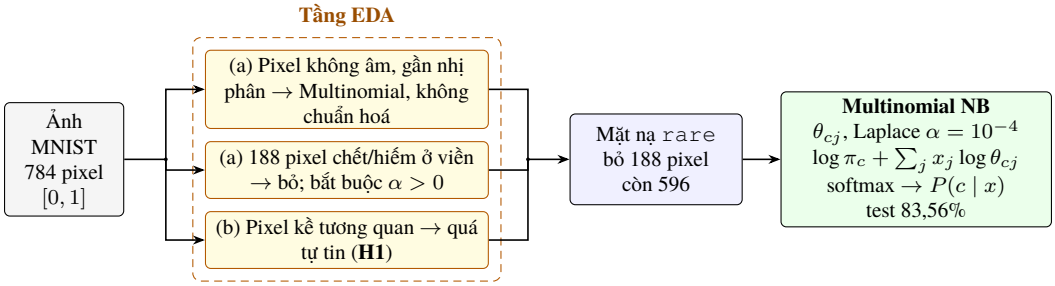
$$\theta_{cj} = \frac{N_{cj} + \alpha}{\sum_{j'} N_{cj'} + \alpha d}, \qquad \log P(c \mid x) \propto \log \pi_c + \sum_j x_j \log \theta_{cj}. \tag{1}$$

Hàm quyết định tuyến tính theo x ($w_{cj} = \log \theta_{cj}$, $b_c = \log \pi_c$), cùng dạng với LR, nên **H1** so hai cách tìm trọng số trên cùng một dạng hàm. $\alpha > 0$ là bắt buộc vì nhiều pixel hiếm có $N_{cj} = 0$. Pipeline cuối ở Hình 3. Để kiểm chứng lựa chọn Multinomial, nhóm cài thêm Gaussian NB, với $\log P(c \mid x) \propto \log \pi_c - \frac{1}{2} \sum_j [\log 2\pi\sigma_{cj}^2 + (x_j - \mu_{cj})^2 / \sigma_{cj}^2]$ và σ_{cj}^2 cộng $\varepsilon = v \cdot \max_j \mathrm{Var}(x_j)$ (cần thiết vì dead pixel có phương sai 0).

Thuật toán 1 Multinomial Naive Bayes

Đầu vào: $X \in \mathbb{R}_{\geq 0}^{N \times d}$, nhãn y, $\alpha > 0$

1: **với** $c = 1..K$ **thực hiện:**
2: $\quad \pi_c \leftarrow |\{i : y_i = c\}|/N; \quad N_{c\cdot} \leftarrow \sum_{i:y_i=c} X_i.$
3: $\quad \log \theta_{c\cdot} \leftarrow \log(N_{c\cdot} + \alpha) - \log(\sum_j N_{cj} + \alpha d)$
4: **dự đoán:** $s \leftarrow X' \log \theta^\top + \log \pi; \quad P \leftarrow \exp(s - \mathrm{logsumexp}(s)); \quad \hat{y} \leftarrow \arg\max_c s_c$



Hình 3: Pipeline của Multinomial NB. Ảnh vào đi qua *tầng EDA*: mỗi ô vàng là một phát hiện EDA (mục 2) phục vụ mô hình này và quyết định mà nó dẫn tới; các nhánh hợp lại thành bước đặc trưng (xanh dương, chỉ học trên train) rồi tới bộ phân lớp (xanh lá).

3.2 Hồi quy softmax (LR)

Với $\tilde{x} = [x; 1]$ và $W \in \mathbb{R}^{(d+1) \times K}$, $p_{ic} = \mathrm{softmax}(W^\top \tilde{x}_i)_c$. Hàm mục tiêu là cross-entropy cộng L2 không phạt bias:

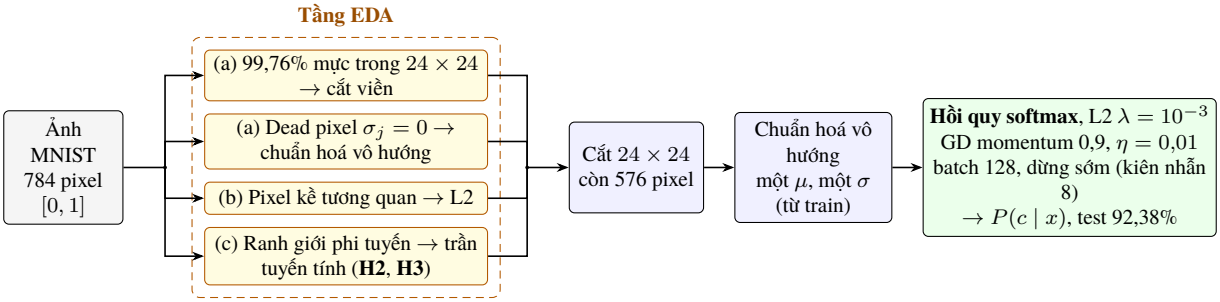
$$J(W) = -\frac{1}{N} \sum_i \log p_{i,y_i} + \frac{\lambda}{2} \|W_{1:d}\|_F^2, \qquad \nabla J = \frac{1}{N} \tilde{X}^\top (P - Y) + \lambda M \odot W, \tag{2}$$

với Y là one-hot và M là mặt nạ bằng 0 ở hàng bias. J lồi nên khởi tạo $W = 0$. Nhóm chọn hồi quy softmax thay cho SVM tuyến tính vì cần xác suất để đo **H1**. Huấn luyện bằng mini-batch gradient descent có momentum, dừng sớm theo val loss (Thuật toán 2); pipeline cuối ở Hình 4.

Thuật toán 2 Mini-batch GD có momentum và dừng sớm (dùng cho LR; MLP dùng cùng vòng lặp với gradient từ lan truyền ngược)

Đầu vào: η, momentum μ, cỡ batch B, kiên nhẫn p, số epoch tối đa E

$W \leftarrow 0$, $V \leftarrow 0$, $L^* \leftarrow \infty$, $t \leftarrow 0$
với $e = 1..E$ **thực hiện:**
  với mỗi mini-batch $\mathcal{B}$ (cỡ B) của một hoán vị ngẫu nhiên tập train **thực hiện:**
    $V \leftarrow \mu V - \eta \nabla J_{\mathcal{B}}(W)$; $\quad W \leftarrow W + V$
  $L \leftarrow$ cross-entropy trên val
  nếu $L < L^*$ **thì**
    $L^* \leftarrow L$, $W^* \leftarrow W$, $t \leftarrow 0$
  ngược lại
    $t \leftarrow t + 1$; **nếu** $t \geq p$ **thì dừng**
trả về W^*



Hình 4: Pipeline của hồi quy softmax (đọc như Hình 3). Phát hiện (c) không thêm bước nào mà cho biết trước giới hạn của mô hình tuyến tính, được kiểm ở **H2** và **H3**.

3.3 Mạng nơ-ron nhiều lớp (MLP)

Với $A^{(0)} = X$, lan truyền thuận $Z^{(l)} = A^{(l-1)} W^{(l)\top} + b^{(l)}$, $A^{(l)} = g(Z^{(l)})$, và softmax ở lớp ra. Lan truyền ngược [5] cho cross-entropy trung bình trên batch:

$$\delta^{(L)} = \tfrac{1}{B}(\hat{Y} - Y), \quad \delta^{(l)} = \big(\delta^{(l+1)} W^{(l+1)}\big) \odot g'(Z^{(l)}), \quad \frac{\partial J}{\partial W^{(l)}} = \delta^{(l)\top} A^{(l-1)} + \lambda W^{(l)}, \quad \frac{\partial J}{\partial b^{(l)}} = \textstyle\sum_i \delta_i^{(l)}. \tag{3}$$

Khởi tạo He cho ReLU, Xavier cho tanh. Dropout dạng inverted [6]: khi huấn luyện, nhân $A^{(l)}$ với mặt nạ Bernoulli rồi chia $1 - p$; khi dự đoán thì giữ nguyên. Tối ưu bằng SGD có momentum với dừng sớm theo val loss, khôi phục trọng số tốt nhất, như Thuật toán 2. Toàn bộ lan truyền thuận và ngược viết bằng NumPy; không dùng autograd khi huấn luyện. Thuật toán 3 là một bước cập nhật trên một mini-batch; vòng lặp epoch, dừng sớm và khôi phục trọng số tốt nhất giống Thuật toán 2. Pipeline và kiến trúc cuối ở Hình 5.

Thuật toán 3 Một bước huấn luyện MLP trên mini-batch $(X_{\mathcal{B}}, Y_{\mathcal{B}})$, $|\mathcal{B}| = B$

Đầu vào: Trọng số $\{W^{(l)}, b^{(l)}\}_{l=1}^{L}$, vận tốc $\{V^{(l)}\}$, η, μ, λ, tỉ lệ dropout p

$A^{(0)} \leftarrow X_{\mathcal{B}}$

với $l = 1..L$ **thực hiện:** ▷ lan truyền thuận, lưu $A^{(l-1)}$ và $Z^{(l)}$

  $Z^{(l)} \leftarrow A^{(l-1)} W^{(l)\top} + b^{(l)}$

  nếu $l < L$ **thì** $A^{(l)} \leftarrow g(Z^{(l)})$; nếu $p > 0$: $A^{(l)} \leftarrow A^{(l)} \odot M^{(l)}/(1-p)$, $M^{(l)} \sim \text{Bernoulli}(1-p)$

$\hat{Y} \leftarrow \text{softmax}(Z^{(L)})$; $\quad \delta^{(L)} \leftarrow (\hat{Y} - Y_{\mathcal{B}})/B$

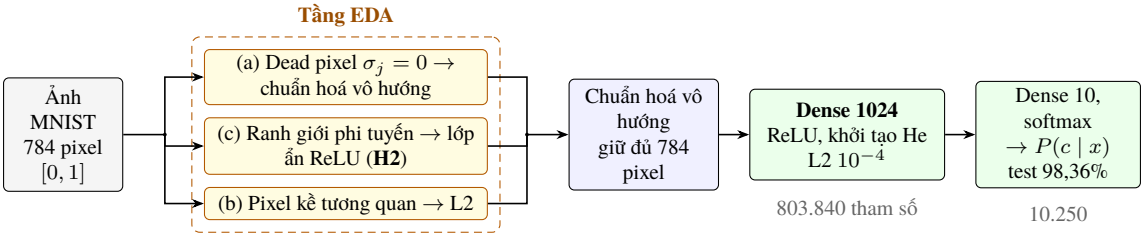
với $l = L..1$ **thực hiện:** ▷ lan truyền ngược

  $\nabla W^{(l)} \leftarrow \delta^{(l)\top} A^{(l-1)} + \lambda W^{(l)}$; $\quad \nabla b^{(l)} \leftarrow \sum_i \delta_i^{(l)}$

  nếu $l > 1$ **thì** $\delta^{(l-1)} \leftarrow \big(\delta^{(l)} W^{(l)}\big) \odot M^{(l-1)}/(1-p) \odot g'(Z^{(l-1)})$ (bỏ M khi $p = 0$)

với $l = 1..L$ **thực hiện:** ▷ SGD có momentum, như Thuật toán 2

  $V^{(l)} \leftarrow \mu V^{(l)} - \eta \nabla W^{(l)}$; $\quad W^{(l)} \leftarrow W^{(l)} + V^{(l)}$ (tương tự cho $b^{(l)}$)



Hình 5: Pipeline của MLP cuối, 784–1024–10 (đọc như Hình 3); số dưới khối là số tham số, tổng 814.090, mỗi nơ-ron lớp 1 nối với từng pixel. Huấn luyện: SGD momentum 0,9, $\gamma = 0{,}05$, batch 128, dừng sớm theo val loss (kiên nhẫn 5), giữ trọng số tốt nhất ở epoch 42.

3.4 Random Forest

Mỗi cây là một cây CART dùng Gini $G = 1 - \sum_c p_c^2$, mọc tới khi lá thuần nhất, trên một mẫu bootstrap. Ở mỗi nút chỉ xét m đặc trưng chọn ngẫu nhiên. Pixel được lượng tử hoá thành n_b mức trước khi đưa vào rừng, nên mỗi pixel chỉ có $n_b - 1$ ngưỡng ứng viên. Rừng dự đoán bằng trung bình xác suất của các cây (soft voting) và ước lượng lỗi bằng out-of-bag (OOB). Rừng là `RandomForestClassifier` của scikit-learn [8] (`bootstrap=True`, `oob_score=True`); phần nhóm tự viết là tiền xử lý (mặt nạ pixel, lượng tử) và các phép phân tích rừng: accuracy theo số cây, sức mạnh và tương quan lỗi giữa các cây, độ quan trọng theo khối. Pipeline cuối ở Hình 6.

Thuật toán 4 Random Forest

Đầu vào: Mã bin $X \in \{0..n_b - 1\}^{N \times d}$, số cây T, số đặc trưng mỗi nút m

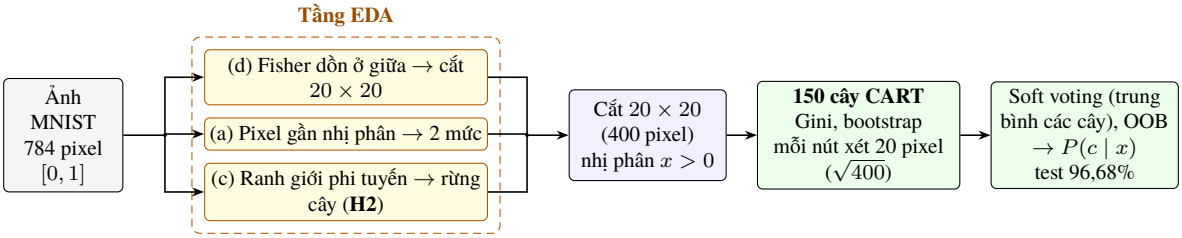
với $t = 1..T$ **thực hiện:**

  Rút bootstrap S_t (cỡ N, có hoàn lại); ghi các mẫu OOB

  Grow(nút gốc với S_t): **nếu** nút thuần nhất **thì** lá ← phân phối lớp

    ngược lại: chọn ngẫu nhiên m đặc trưng; lấy ngưỡng trên một đặc trưng giảm Gini nhiều nhất; chia nút; Grow hai nút con

trả về $P(c \mid x) = \frac{1}{T} \sum_t P_t(c \mid x)$

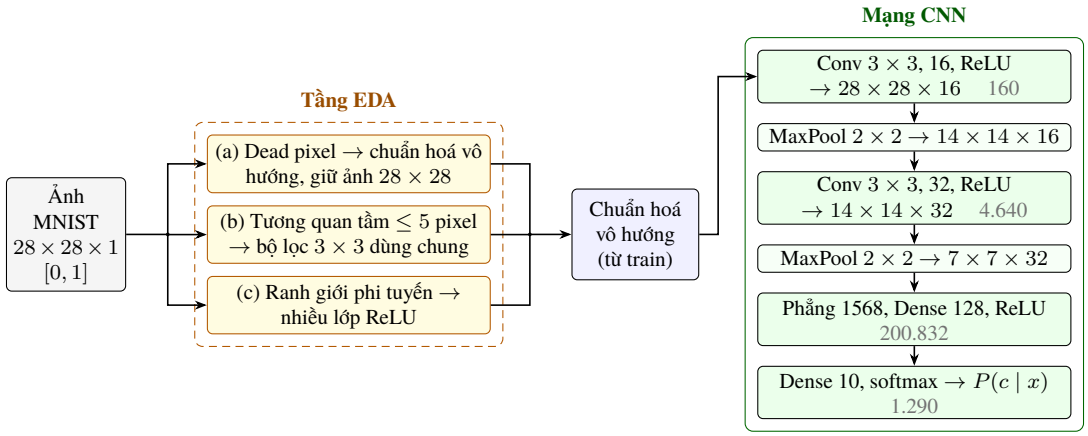


Hình 6: Pipeline của Random Forest (đọc như Hình 3). Mặt nạ và số mức lượng tử chọn ở E1–E2 (mục 4); rừng là `RandomForestClassifier` của scikit-learn, bước đặc trưng do nhóm tự viết.

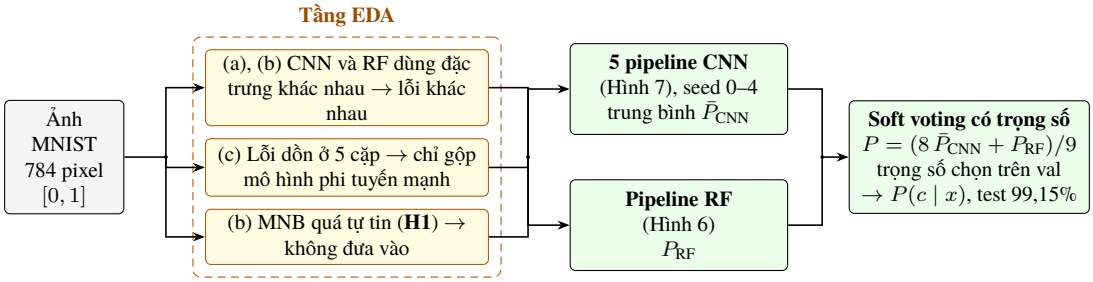
3.5 Phần nâng cao: CNN và tổ hợp mô hình

CNN. Hai khối Conv 3×3 + ReLU + max-pool 2×2 (16 rồi 32 bộ lọc), rồi Dense 128 và Dense 10 (Hình 7), tổng 206.922 tham số, ít hơn MLP gần 4 lần vì mỗi bộ lọc dùng chung cho mọi vị trí. Tích chập cài bằng im2col trên bố cục NHWC, lan truyền ngược tự viết, tối ưu Adam [7]. CNN được thêm vào vì phát hiện (b): bộ lọc nhỏ dùng chung cho mọi vị trí đủ bao tầm tương quan (khoảng 5 pixel), trong khi lớp đầu của MLP phải học riêng trọng số cho từng pixel.

Tổ hợp. Soft voting có trọng số $P = \sum_k w_k P_k / \sum_k w_k$ trên các mô hình cuối đã có, gồm cả *seed ensemble*: 5 CNN cùng cấu hình, chỉ khác seed (khởi tạo và thứ tự mini-batch). Tổ hợp và trọng số chọn trên val; pipeline của phương án cuối ở Hình 8.



Hình 7: Pipeline và kiến trúc CNN cuối (đọc như Hình 3); số xám cuối mỗi lớp là số tham số, tổng 206.922, ít hơn MLP gần 4 lần. Tích chập dùng padding 1 nên giữ kích thước; max-pool không có tham số. Huấn luyện: Adam $\eta = 10^{-3}$, batch 128, tối đa 20 epoch, dừng sớm theo val loss (kiên nhẫn 3). Test 98,82%.



Hình 8: Pipeline của tổ hợp cuối (đọc như Hình 3): mỗi thành viên giữ nguyên pipeline của nó (đặc trưng riêng, học trên train), chỉ xác suất đầu ra được gộp. Trung bình 5 CNN khác seed (seed ensemble) giảm phương sai do khởi tạo và thứ tự mini-batch.

3.6 Kiểm chứng tính đúng của bản tự cài

Gradient của LR và CNN được so với sai phân trung tâm; lan truyền ngược của MLP so với autograd của PyTorch (sai số tương đối $< 10^{-8}$); tích chập, max-pool và Adam so với `torch`; các hàm phân tích rừng so với cách tính vét cạn trên rừng sklearn; MNB và Gaussian NB khớp `sklearn` tới 10^{-10}; nghiệm tối ưu của LR khi chạy đủ lâu khớp `LogisticRegression`. Tổng cộng 25 kiểm tra tự động, phần lớn so mã của nhóm với một lời giải độc lập.

4 Thí nghiệm

4.1 Thiết lập chung

Chọn cấu hình. Mỗi thí nghiệm (E1, E2, …) đổi một yếu tố so với cấu hình đang có và chọn theo validation. Khi chênh lệch nhỏ hơn mức dao động (10 ảnh val với các mô hình chỉ chạy một lần: MNB, LR, MLP, CNN; 2 lần độ lệch chuẩn theo seed với RF), chọn phương án đơn giản hơn: ít đặc trưng hơn hoặc nhanh hơn. Test chỉ dùng

một lần cho mô hình cuối. Mọi con số dưới đây là trên validation (`results/<mô hình>/log.txt`); bảng đầy đủ và vị trí mã nguồn ở `docs/<mô hình>_pipeline.md`.

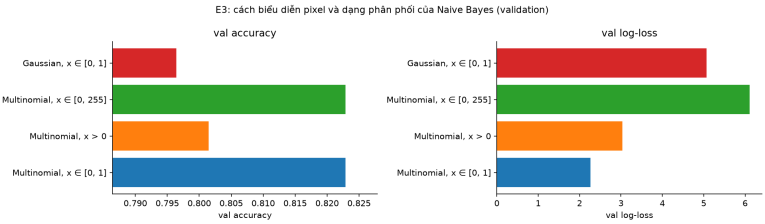
Độ đo. Accuracy; precision, recall, F1 macro; ma trận nhầm lẫn trên train, val và test. Với mô hình có xác suất, thêm log-loss, tỉ lệ ảnh bị đoán sai với xác suất $> 0{,}99$, và độ tự tin trung bình (trung bình xác suất lớn nhất): lớn hơn accuracy là quá tự tin, nhỏ hơn là thiếu tự tin.

Thời gian. *Huấn luyện*: thời gian `fit` của cả pipeline cuối trên 50.000 ảnh, gồm tiền xử lý và val loss cho dừng sớm. *Dự đoán*: 10.000 ảnh test. Máy: CPU Intel 14 lõi, Windows 11, Python 3.14, NumPy 2.x; số luồng ở Bảng 2. Thời gian chỉ để so độ lớn tương đối.

Độ quan trọng. Mọi mô hình dùng chung một số đo: accuracy val giảm bao nhiêu khi hoán vị cùng lúc một khối 4×4 pixel giữa các ảnh (49 khối, 2 lần lặp). Hoán vị theo khối vì pixel kề bên mang cùng thông tin (phát hiện b), hoán vị từng pixel sẽ đánh giá thấp độ quan trọng. Tâm gần nhất (val 0,8048) là mốc dưới trong mọi bảng.

4.2 Multinomial Naive Bayes

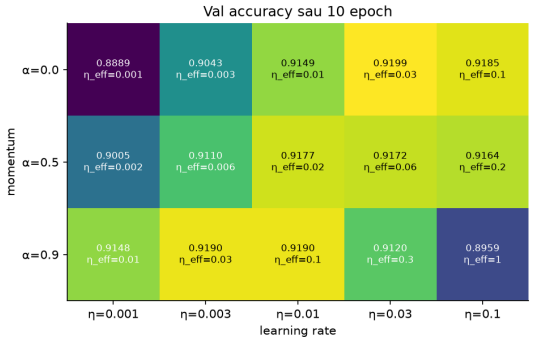
- **E1, hệ số làm trơn** $\alpha \in \{10^{-4}, \ldots, 10\}$: val accuracy từ 0,8205 tới 0,8228, chênh 23 ảnh. *Chọn* $\alpha = 10^{-4}$ (cao nhất). Làm trơn gần như không quan trọng: giới hạn của MNB nằm ở giả định độc lập, không ở ước lượng θ.
- **E2, bỏ pixel**: bỏ pixel chết, pixel hiếm hay cắt 24×24 đều cho 0,8218–0,8229; cắt 20×20 tụt còn 0,8126. *Chọn* mặt nạ `rare` (596 pixel): ít đặc trưng nhất trong các mặt nạ cách tốt nhất không quá 0,001.
- **E3, biểu diễn** (Hình 9): cường độ $[0, 1]$ hơn nhị phân 2,1 điểm. Nhân x với 255 không đổi accuracy nhưng log-loss tăng từ 2,26 lên 6,11: độ tự tin của NB phụ thuộc thang đo. Gaussian NB kém 2,7 điểm (0,7964), log-loss gấp 2,2 lần, 87,4% số lỗi có $p > 0{,}99$; với ε mặc định của sklearn (10^{-9}) chỉ đạt 0,5849. *Giữ* Multinomial trên $[0, 1]$ như EDA đã chọn.



Hình 9: MNB, E3: accuracy và log-loss trên val theo cách biểu diễn pixel, so với Gaussian NB.

4.3 Hồi quy softmax

- **E1, chuẩn hoá**: không chuẩn hoá 0,9155; chuẩn hoá vô hướng (một μ, một σ) 0,9210; z-score từng pixel 0,9182, vì pixel gần chết có σ rất nhỏ nên $|z|$ trên val lên tới 1314. *Chọn* chuẩn hoá vô hướng.
- **E2a, learning rate** $\times$ **momentum** (Hình 10): bước hiệu dụng $\eta/(1-\mu)$ mới là đại lượng quyết định; $\eta_{\text{eff}} \approx 0{,}02$–$0{,}1$ tốt nhất, $\eta_{\text{eff}} = 1$ sụt về 0,896. *Chọn* $\mu = 0{,}9$, $\eta = 0{,}01$ (val loss nhỏ nhất, 0,2919).

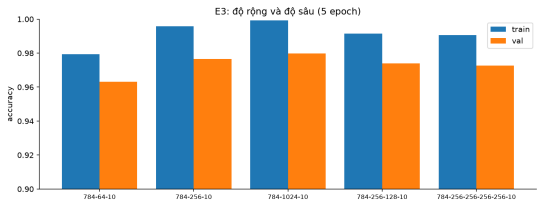


Hình 10: Hồi quy softmax, E2a: accuracy val trên lưới learning rate $\eta\times$ momentum μ (10 epoch).

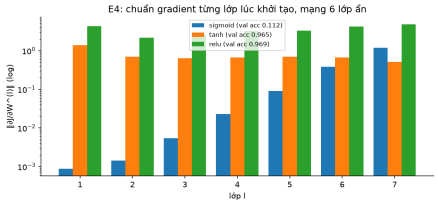
- **E2b–c, batch và tốc độ hội tụ**: sau 10 epoch mini-batch 128 đã có $J = 0{,}258$, full-batch cần 100 epoch mới xuống 0,257 (có momentum). Mỗi batch được dò η riêng; full-batch chỉ đạt 0,8994 sau 20 epoch. *Chọn* batch 128: không batch nào khác cách nó dưới 0,002.
- **E3, L2**: λ từ 0 tới 10^{-3} giữ accuracy 0,920–0,922, còn $\lambda = 10^{-2}$ thiếu khớp (0,9155). *Chọn* $\lambda = 10^{-3}$: val loss nhỏ nhất (0,2872), $\|W\|$ giảm từ 11,1 xuống 6,7, ảnh trọng số bớt lốm đốm.
- **E4, bỏ pixel**: *chọn* cắt 24×24 (576 pixel), ít đặc trưng nhất trong các mặt nạ có val loss cách tốt nhất dưới 0,002. Mô hình cuối dừng sớm (kiên nhẫn 8, tối đa 150 epoch): dừng ở epoch 21, giữ epoch 13.

4.4 MLP

- **E0, mốc**: 784–256–128–10, ReLU, He, SGD $\gamma = 0{,}05$, momentum 0,9, batch 128 đạt 0,9766 sau 10 epoch (Lab 04 yêu cầu $\geq 0{,}97$). E1–E6 mỗi lần đổi một yếu tố so với mốc này, chạy 5 epoch.
- **E1, learning rate × momentum**: $\gamma_{\text{eff}} = \gamma/(1-\alpha) \geq 5$ phân kỳ, $\gamma_{\text{eff}} = 1$ đã kém. *Chọn* $\gamma = 0{,}05$, $\alpha = 0{,}9$ (0,9740), trùng với mốc.
- **E2, batch**: 256 đạt 0,9742, hơn 128 đúng 2 ảnh; batch 16 với cùng γ thì không học được (0,099). Theo luật val accuracy cao nhất, E2 chọn 256, nhưng xem E7.
- **E3, độ rộng và độ sâu** (Hình 11a): rộng tốt hơn sâu khi cùng số epoch: 784–1024–10 đạt 0,9798, mạng 4 lớp ẩn 256 chỉ 0,9726. *Chọn* một lớp ẩn 1024.
- **E4–E5, kích hoạt và khởi tạo** (Hình 11b): với 6 lớp ẩn và sigmoid, chuẩn gradient ở lớp 1 nhỏ hơn lớp cuối 1355 lần, mạng không học (0,112); tanh và ReLU giữ gradient đều giữa các lớp. Khởi tạo hằng làm 64 nơ-ron chỉ còn 5 nơ-ron khác nhau (đối xứng không bị phá). Với ReLU, khởi tạo He giữ độ lệch chuẩn đầu ra ở lớp 10 là 0,95, Xavier chỉ còn 0,03. *Giữ* ReLU + He.
- **E6, điều chuẩn** (15 epoch): L2 10^{-4} đạt 0,9818 (val loss 0,0817), dropout 0,2 đạt 0,9770, không điều chuẩn 0,9769 và val loss tăng dần. *Chọn* L2 10^{-4}, không dropout.
- **E7, ứng viên cuối** (tối đa 50 epoch, dừng sớm kiên nhẫn 5): mỗi thí nghiệm chỉ đổi một yếu tố, nên ghép mọi lựa chọn chưa chắc tốt nhất. Ứng viên A (gộp E1, E2, E3, E6) và B (mốc + E3 + E6) chỉ khác nhau ở batch 256 và 128: A tốt nhất ở epoch 9 với 0,9802 (dừng ở epoch 14), B học tới epoch 42 và đạt 0,9831. Lợi thế 2 ảnh của batch 256 sau 5 epoch không còn khi huấn luyện đủ lâu. *Chọn* B (batch 128); ứng viên C (mốc + E6) đạt 0,9733.



(a) E3: độ rộng và độ sâu (5 epoch).



(b) E4: chuẩn gradient theo lớp, 6 lớp ẩn.

Hình 11: Thí nghiệm siêu tham số của MLP (validation).

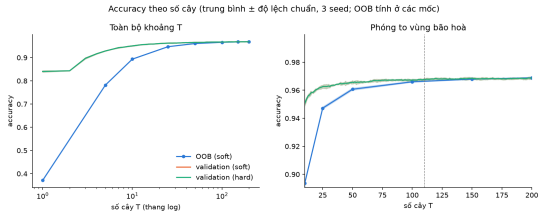
4.5 Random Forest

RF có ngẫu nhiên: đường theo số cây (E3) chạy 3 seed và phép xác nhận ở E4 chạy 2 seed, báo trung bình $\pm$ độ lệch chuẩn; các lưới còn lại chạy một seed. Một cây CART đầy đủ (`DecisionTreeClassifier`) thuộc lòng train (1,0000) nhưng chỉ đạt 0,8784 trên val.

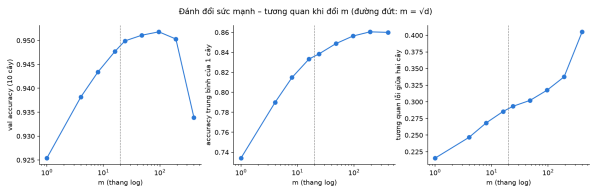
- **E1, số ngưỡng mỗi pixel**: 2, 4, 8, 16 và 256 mức cho 0,9588–0,9608, trong khi độ lệch chuẩn theo seed là 0,0009. *Chọn* 2 mức (pixel > 0, "có mực"): ít mức nhất trong khoảng 2 độ lệch chuẩn của mức tốt nhất, OOB cao nhất, và nhanh hơn 256 mức khoảng 1,35 lần.
- **E2, bỏ pixel**: mọi mặt nạ nằm trong khoảng dao động (0,9605–0,9632). *Chọn* cắt 20×20 (400 pixel): ít đặc trưng nhất, thời gian huấn luyện còn 65% so với giữ cả 784 pixel.
- **E3, số cây** (Hình 12a): accuracy tăng nhanh rồi bão hoà; từ $T = 110$, val nằm trong 0,1 điểm của giá trị tại $T = 200$. *Chọn* $T = 150$ (làm tròn lên bội số của 50). OOB đánh giá thấp khi T nhỏ ($-1{,}54$ điểm

ở $T = 25$), vì mỗi mẫu OOB chỉ được khoảng $0{,}37\,T$ cây bỏ phiếu. Soft và hard voting cho đúng cùng accuracy: với pixel nhị phân và cây đầy đủ, hầu hết lá thuần nhất.

- **E4, số pixel xét ở mỗi nút** m (Hình 12b): tăng m làm từng cây mạnh hơn nhưng các cây giống nhau hơn; với $m = d$ (bagging thuần), tương quan lỗi giữa hai cây lên 0,41 và rừng kém đỉnh 1,8 điểm. Ở 10 cây $m = 96$ cao nhất, nhưng khi kiểm lại ở 50 cây, 2 seed, $m = 96$ kém $m = \sqrt{d} = 20$ 0,12 điểm và chậm hơn 2,4 lần. *Chọn* $m = \sqrt{d}$.
- **E5, giới hạn cây**: `min_samples_leaf` $= 1$ tốt nhất (0,9618, tăng lên 30 còn 0,9287); `max_depth` 20 hơn cây đầy đủ 0,14 điểm, dưới mức dao động. *Giữ* cây đầy đủ như cách dựng RF chuẩn: phần dao động của từng cây được trung bình đi.



(a) E3: accuracy val và OOB theo số cây (3 seed).



(b) E4: sức mạnh và tương quan cây theo m.

Hình 12: Thí nghiệm siêu tham số của Random Forest (validation).

4.6 CNN và tổ hợp

- **E1, learning rate của Adam** (3 epoch): $3 \cdot 10^{-4}$ đạt 0,9788, 10^{-3} đạt 0,9837, $3 \cdot 10^{-3}$ đạt 0,9809. *Chọn* 10^{-3}.
- **E2, số bộ lọc**: (8, 16) đạt 0,9816 trong 58 giây, (16, 32) đạt 0,9837 trong 101 giây, (32, 64) đạt 0,9800 trong 185 giây. *Chọn* (16, 32): nhanh nhất trong các cấu hình cách tốt nhất không quá 0,001.
- **E3, số epoch**: tối đa 20, dừng sớm khi val loss không giảm 3 epoch. Val loss nhỏ nhất ở epoch 5 (0,0493); dừng ở epoch 8 và khôi phục trọng số epoch 5.
- **Tổ hợp**: thành viên và trọng số được chọn trên val (mục 5.7): 5 CNN (seed 0–4) trọng số 8 cộng RF trọng số 1, soft voting.

5 Kết quả

5.1 So sánh tổng hợp

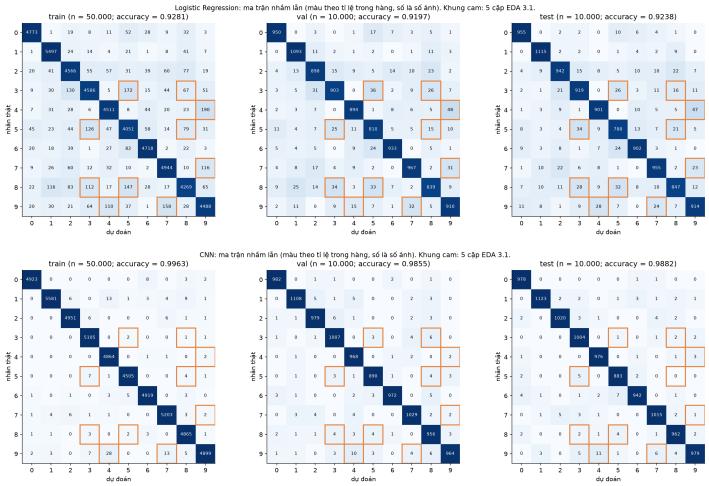
Bảng 2 là kết quả của các mô hình cuối; thứ tự giống nhau trên val và test. Khoảng cách train–val phản ánh dung lượng mô hình: MNB và LR gần như không overfit (giới hạn nằm ở dạng hàm); RF và MLP thuộc lòng train (100%) nhưng vẫn tổng quát tốt nhờ trung bình nhiều cây (RF) và L2 cùng dừng sớm (MLP). Hình 13 so ma trận nhầm lẫn của hai đầu phép so sánh: LR, mô hình tuyến tính tốt nhất và đối tượng của **H1**, **H3**, và CNN, mô hình đơn tốt nhất; đặt cạnh nhau, chúng cho thấy lỗi ở 5 cặp tâm gần (khung cam) giảm thế nào khi bỏ được giới hạn tuyến tính (ma trận của các mô hình khác ở `results/<mô hình>/figures/`). Theo Bảng 3, ở mọi mô hình P, R, F1 macro chênh nhau không quá 1,2 điểm (lớn nhất ở MNB: P 0,832, R 0,820) vì 10 lớp gần cân bằng, nên xếp hạng theo F1 macro trùng với xếp hạng theo accuracy trên cả ba tập.

Bảng 2: Kết quả mô hình cuối: accuracy, log-loss trên test, thời gian huấn luyện (giây, 50.000 ảnh), thời gian dự đoán (giây, 10.000 ảnh test) và số luồng tính toán khi đo (tổ hợp: CNN 8 luồng, RF 14 luồng). P, R, F1 ở Bảng 3.

Mô hình	Accuracy			Log-loss	Huấn luyện	Dự đoán	Luồng
	train	val	test	test	(s)	(s)	
Tâm gần nhất (mốc)	0,8080	0,8048	0,8198	—	0,2	0,02	1
Multinomial NB	0,8247	0,8229	0,8356	2,007	0,5	0,05	1
Hồi quy softmax	0,9281	0,9197	0,9238	0,276	2,7	0,07	8
Random Forest	1,0000	0,9694	0,9668	0,218	7,3	0,15	14
MLP	1,0000	0,9831	0,9836	0,054	247,7	0,14	8
CNN	0,9963	0,9855	0,9882	0,039	273,1	2,88	8
5 CNN + RF (soft voting)	0,9993	0,9913	**0,9915**	0,041	1452,4	14,51	8 / 14

Bảng 3: Precision, recall và F1 trung bình macro trên train, val và test.

Mô hình	Train			Val			Test		
	P	R	F1	P	R	F1	P	R	F1
Tâm gần nhất (mốc)	0,8116	0,8046	0,8059	0,8103	0,8014	0,8031	0,8230	0,8167	0,8178
Multinomial NB	0,8317	0,8216	0,8233	0,8317	0,8200	0,8220	0,8431	0,8325	0,8342
Hồi quy softmax	0,9274	0,9270	0,9271	0,9191	0,9188	0,9188	0,9229	0,9226	0,9227
Random Forest	1,0000	1,0000	1,0000	0,9693	0,9693	0,9693	0,9666	0,9665	0,9666
MLP	1,0000	1,0000	1,0000	0,9830	0,9830	0,9830	0,9835	0,9835	0,9835
CNN	0,9963	0,9963	0,9963	0,9854	0,9855	0,9854	0,9881	0,9882	0,9881
5 CNN + RF (soft voting)	0,9993	0,9993	0,9993	0,9913	0,9913	0,9913	0,9915	0,9915	0,9915



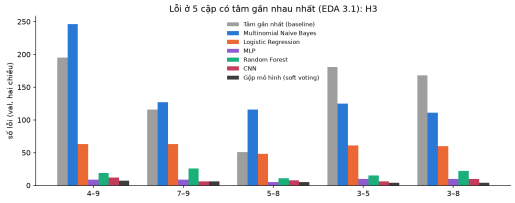
Hình 13: Ma trận nhầm lẫn trên train | val | test của hồi quy softmax (trên) và CNN (dưới). Khung cam: 5 cặp tâm gần của EDA.

5.2 Độ tin cậy của xác suất (H1)

LR vượt MNB 9,7 điểm accuracy trên val (0,9197 so với 0,8229). Khác biệt về xác suất còn rõ hơn: log-loss 0,291 so với 2,259, và tỉ lệ ảnh bị đoán sai với $p > 0{,}99$ là 0,19% so với 10,35%. Độ tự tin trung bình của MNB là 0,978 trong khi accuracy chỉ 0,823; của LR là 0,905, sát accuracy 0,920. Đây đúng là hệ quả của việc cộng bằng chứng từ các pixel tương quan như thể độc lập, và tăng α không sửa được. **H1** được ủng hộ. Ngược lại, RF *thiếu* tự tin (độ tự tin trung bình 0,845 trong khi accuracy 0,969): trung bình xác suất của nhiều cây kéo xác suất về giữa, nên RF không có ảnh nào bị đoán sai với $p > 0{,}99$.

5.3 Mô hình phi tuyến (H2) và lỗi theo cặp lớp (H3)

RF (0,9694) và MLP (0,9831) đều vượt LR (0,9197) trên val, cả về accuracy lẫn F1 macro: **H2** được ủng hộ. Bảng 4 và Hình 14 cho lỗi ở 5 cặp tâm gần. Với MNB và LR, 5 cặp chiếm 40,9% và 36,7% tổng lỗi (mức rải đều là 11,1%), và cả 5 cặp đều nằm trong top 5 cặp nhầm nhiều nhất: **H3**(a) đạt. So với LR, RF giảm 68% lỗi ở 5 cặp so với 62% tổng lỗi, MLP giảm 85% so với 79%: **H3**(b) đạt. Sau khi giảm, 5 cặp còn chiếm 30,4% lỗi của RF, 25,4% của MLP và 29,0% của CNN, và chỉ 3/5 cặp còn trong top 5, nhường chỗ cho các cặp khác (2–3, 2–7 với RF và MLP; 8–9, 2–3 với CNN). Như vậy "khó" theo khoảng cách tâm là khó nhất với mô hình tuyến tính; MLP giảm phần khó này rõ nhất.



Hình 14: Lỗi ở 5 cặp tâm gần trên val, cộng hai chiều (**H3**).

Bảng 4: Lỗi trên val ở 5 cặp tâm gần (cộng hai chiều) và tỉ lệ trong tổng lỗi.

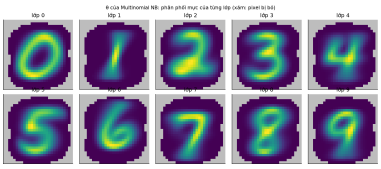
Mô hình	4–9	7–9	5–8	3–5	3–8	Tổng lỗi	Tỉ lệ ở 5 cặp
Tâm gần nhất	195	116	51	181	168	1952	36,4%
Multinomial NB	246	127	116	125	111	1771	40,9%
Hồi quy softmax	63	63	48	61	60	803	36,7%
Random Forest	19	26	11	15	22	306	30,4%
MLP	9	9	5	10	10	169	25,4%
CNN	12	6	8	6	10	145	29,0%
5 CNN + RF	7	6	5	4	4	87	29,9%

5.4 Khả năng giải thích (H4)

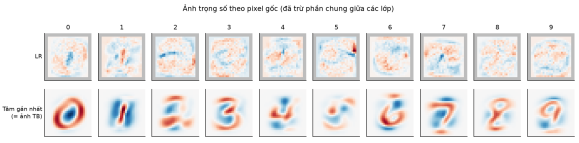
Hình 15 đặt độ quan trọng theo khối của mọi mô hình cạnh tổng Fisher của khối. Hệ số Spearman [4] là 0,90 (MNB), 0,91 (LR), 0,88 (MLP), 0,88 (RF) và 0,93 (CNN), đều $\geq 0{,}7$: **H4** được ủng hộ. Khối quan trọng nhất của mọi mô hình nằm ở giữa ảnh; hoán vị khối đó làm accuracy val giảm 3,6 (MNB) đến 8,5 điểm (LR). Mỗi mô hình còn có cách giải thích riêng với độ trực tiếp khác nhau (Hình 16): θ_c của MNB chính là ảnh mực trung bình của lớp, đọc được ngay hình chữ số; ảnh trọng số của LR lốm đốm vì pixel kề nhau tương quan mạnh nên trọng số chia giữa chúng gần như tuỳ ý, chỉ vài nét nổi lên (vạch ngang của 2, góc của 5), khó đọc hơn nhiều so với ảnh của tâm gần nhất; MDI của RF tập trung ở giữa khung 20×20, giống tần suất có mực và hoán vị khối; trọng số lớp 1 của MLP là các bộ phát hiện nét cục bộ (vạch ngang, vạch chéo), không gắn với lớp nào, nên chỉ cho biết mạng nhìn vào đâu. CNN chỉ có số đo hoán vị.



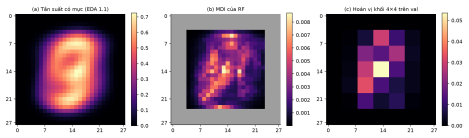
Hình 15: Độ quan trọng theo khối 4×4 (accuracy val giảm khi hoán vị khối), cùng một cách đo cho mọi mô hình; ô đầu tiên là tổng Fisher theo khối.



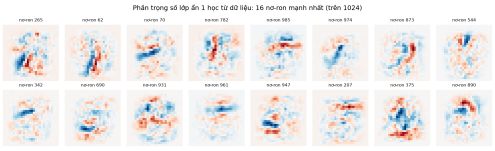
(a) MNB: θ_c của 10 lớp.



(b) LR: ảnh trọng số theo lớp (trên) và của tâm gần nhất (dưới).



(c) RF: tần suất có mực, MDI và hoán vị khối.

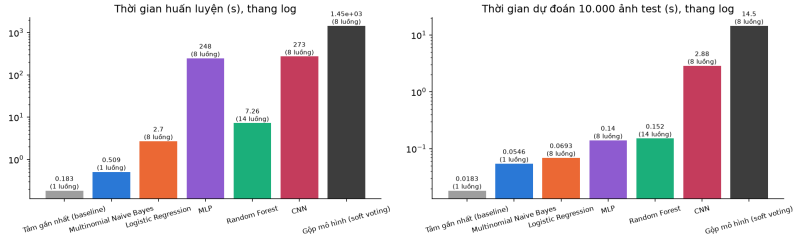


(d) MLP: trọng số lớp 1 của 16 nơ-ron mạnh nhất.

Hình 16: Cách giải thích riêng của từng mô hình bắt buộc.

5.5 Chi phí tính toán

Thời gian huấn luyện trải gần ba bậc độ lớn (Hình 17), từ 0,5 giây (MNB, một lần đếm) tới 273 giây (CNN). MLP (47 epoch) và CNN (8 epoch, khoảng 34 giây mỗi epoch) tốn gần bằng nhau vì hai lý do khác nhau: MLP cần nhiều epoch, còn mỗi epoch của CNN đắt. Khi dự đoán, RF duyệt 150 cây nhưng nhanh ngang MLP (0,15 so với 0,14 giây), còn CNN ít tham số hơn MLP 4 lần nhưng chậm hơn khoảng 20 lần, vì mỗi bộ lọc được áp lên mọi vị trí của ảnh. Các con số phụ thuộc số luồng (Bảng 2), chỉ nên so theo bậc độ lớn.



Hình 17: Thời gian huấn luyện (trái) và dự đoán 10.000 ảnh test (phải), thang log, kèm số luồng.

5.6 Đối chiếu với thư viện

Bảng 5 huấn luyện lại mỗi mô hình tự cài bằng scikit-learn [8] hoặc PyTorch [9] với cùng đặc trưng và siêu tham số. MNB khớp tuyệt đối, MLP chênh 0,13 điểm, CNN kém bản PyTorch 0,06 điểm (khác khởi tạo và dãy trộn; phần đúng đã kiểm bằng test gradient). Với LR, bản đối chiếu là `MLPClassifier` không lớp ẩn với cùng SGD, momentum, L2 và 13 epoch, vì `LogisticRegression` không có mini-batch hay dừng sớm; hai bản chênh 0,71 điểm trên test vì với momentum 0,9 nghiệm sau từng epoch còn dao động quanh cực tiểu. Giải tới hội tụ bằng lbfgs (`LogisticRegression`, $C = 1/(\lambda N)$) cho test 0,9238, đúng bằng bản tự cài: dừng sớm theo val loss không làm mất accuracy. Về thời gian, MNB và MLP tự cài nhanh hơn sklearn (MLP hơn 2 lần), LR ngang nhau, CNN NumPy chậm hơn PyTorch khoảng 3 lần (cùng 8 luồng).

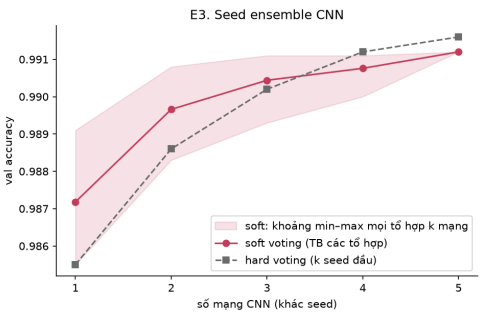
Bảng 5: Đối chiếu với thư viện: accuracy test và thời gian huấn luyện (giây), cùng đặc trưng và siêu tham số.

Mô hình	Bản thư viện	Tự cài	Thư viện	Chênh	Huấn luyện (s): tự cài / thư viện
Multinomial NB	`MultinomialNB`	0,8356	0,8356	+0,0000	0,5 / 2,9
Hồi quy softmax	`MLPClassifier`, không lớp ẩn	0,9238	0,9167	+0,0071	2,7 / 7,0
MLP	`MLPClassifier`	0,9836	0,9823	+0,0013	247,7 / 574,4
CNN	`torch.nn`	0,9882	0,9888	−0,0006	273 / 84

Chọn RF thay vì boosting. Boosting phải học tuần tự, còn các cây của RF độc lập nên trồng song song, có sẵn OOB, và accuracy theo số cây chỉ tăng rồi bão hoà. Cái giá của lựa chọn này (sklearn, cùng đặc trưng, 150 vòng): HistGradientBoosting đạt test 0,9781 (hơn RF 1,1 điểm) nhưng huấn luyện lâu hơn 10 lần (150 so với 7,3 giây); AdaBoost (SAMME, cây sâu 3) chỉ đạt 0,9141.

5.7 Phần nâng cao: CNN và tổ hợp

CNN vượt MLP 0,24 điểm trên val với số tham số ít hơn 4 lần; CNN cuối (seed 0) lại là seed kém nhất trong 5 seed (0,9855–0,9891, trung bình 0,9872), nên khoảng cách điển hình với MLP gần 0,4 điểm. Phần lỗi giảm nằm ngoài 5 cặp tâm gần (tổng lỗi -14%, ở 5 cặp chỉ -2%). Gộp các mô hình cổ điển với trọng số bằng nhau không có lợi: không tổ hợp nào hơn thành viên tốt nhất, kể cả RF + MLP (kém MLP 5 ảnh); MNB sai rất khác CNN (tỉ lệ cùng sai khoảng 5%) nhưng quá tự tin nên kéo trung bình về phía sai. Lợi chắc chắn đến từ seed ensemble (Hình 18): 5 CNN giảm lỗi test từ 118 xuống 85 ảnh (-28%), lợi ích giảm dần theo số mạng. Phương án chọn trên val (5 CNN trọng số 8 cộng RF trọng số 1) đạt 0,9915 trên test, bằng đúng 5 CNN một mình (cùng 85 ảnh sai); trên val nó chỉ hơn 1 ảnh, trong mức dao động, và RF làm log-loss test kém đi (0,0410 so với 0,0249). Vì vậy demo dùng 5 CNN không kèm RF ("CNN × 5").



Hình 18: Seed ensemble CNN: accuracy val theo số mạng gộp (soft và hard voting).

6 Thảo luận

Bảng 6 tóm tắt ưu và nhược điểm của bốn phương pháp bắt buộc theo bốn tiêu chí của đề.

Bảng 6: Ưu và nhược điểm của bốn phương pháp bắt buộc.

	Multinomial NB	Hồi quy softmax	MLP	Random Forest
Độ chính xác	Thấp (83,6%), chỉ hơn tâm gần nhất 1,6 điểm; xác suất quá tự tin	Trung bình (92,4%); độ tự tin sát accuracy; trần do ranh giới tuyến tính	Cao nhất (98,4%); độ tự tin sát accuracy (0,988 so với 0,983)	Cao (96,7%); xác suất thiếu tự tin, không lỗi nào có $p > 0{,}99$
Huấn luyện	Nhanh nhất (một lần đếm, 0,5 s); không cần lặp	Nhanh (2,7 s); hàm lồi, ít siêu tham số	Chậm (248 s); nhiều siêu tham số, nhạy với lr/momentum, khởi tạo	Nhanh (7,3 s, 14 luồng); song song tự nhiên; gần như không cần tinh chỉnh
Dự đoán	Rất nhanh (một phép nhân ma trận)	Rất nhanh	Nhanh (0,14 s)	Chậm nhất trong bốn (0,15 s, duyệt 150 cây); mô hình nặng (khoảng 220 MB)
Giải thích	Trực tiếp: θ_c là ảnh mực của lớp	Có ảnh trọng số theo lớp, nhưng lốm đốm vì pixel kề nhau tương quan; khó đọc hơn θ_c	Gián tiếp: chỉ có trọng số lớp 1 và số đo hoán vị	Trung bình: MDI theo pixel, đường đi của từng cây nhưng 150 cây khó đọc

Ba giới hạn khác nhau, đều được EDA dự báo. MNB bị giới hạn bởi *giả định*: đổi α hay mặt nạ pixel không giúp gì, và cộng bằng chứng không độc lập làm nó quá tự tin. LR bị giới hạn bởi *dạng hàm*: lỗi dồn vào các cặp lớp mà ranh giới tuyến tính không tách được, và RF, MLP giảm chọn lọc đúng các lỗi ấy (**H3**b). Khoảng cách RF–MLP (1,4 điểm trên val) nằm ở cách dùng đặc trưng: mỗi phép chia của cây chỉ nhìn một pixel nhị phân, còn MLP tổ hợp nhiều pixel ngay ở lớp đầu.

Accuracy không đủ để chọn mô hình. MNB và LR khác nhau về chất lượng xác suất còn nhiều hơn về accuracy; MLP (hơi quá tự tin) và RF (thiếu tự tin) có accuracy gần nhau nhưng lệch theo hai hướng ngược nhau. Khi xác suất dùng để ra quyết định (ví dụ chuyển ảnh mơ hồ cho người kiểm tra), log-loss và tỉ lệ ảnh sai với xác suất rất cao quan trọng không kém accuracy.

Hạn chế. (i) LR, MLP và MNB chỉ chạy một seed, nên chênh lệch dưới khoảng 0,2 điểm giữa các mô hình mạnh chưa được kiểm định thống kê. (ii) Thời gian đo trên một máy với số luồng khác nhau, chỉ so được theo bậc độ lớn. (iii) Tổ hợp được chọn trong khoảng 40 cấu hình trên val nên val accuracy của nó lạc quan; khoảng cách với 5 CNN một mình nằm trong dao động. (iv) Không tăng cường dữ liệu nên CNN không bất biến với xoay hay dịch lớn; demo bù bằng chuẩn hoá ảnh vẽ, nhưng chữ nghiêng hoặc nét quá mảnh vẫn có thể bị đoán sai. Hướng mở rộng: tăng cường dữ liệu (xoay, dịch, nhiễu) cho CNN và thử trên Fashion-MNIST.

7 Kết luận

Bốn bộ phân lớp bắt buộc cho thứ tự accuracy trên test là MNB (83,6%) < LR (92,4%) < RF (96,7%) < MLP (98,4%); các bản tự cài khớp bản thư viện cùng cấu hình (MNB, MLP trong 0,13 điểm; LR bằng đúng nghiệm hội tụ lbfgs). Cả bốn giả thuyết rút từ EDA đều được ủng hộ: giả định độc lập làm MNB vừa kém chính xác vừa quá tự tin; ranh giới tuyến tính dồn lỗi vào các cặp lớp có tâm gần nhau, và mô hình phi tuyến giảm chọn lọc các lỗi đó; mọi mô hình dựa vào cùng vùng giữa ảnh nơi Fisher score cao. CNN và seed ensemble đưa accuracy test lên 98,8% và 99,2%, nhưng mỗi bước tăng accuracy đều tốn thêm chi phí huấn luyện, dự đoán và khả năng giải thích. Bài học chính là phân tích dữ liệu trước khi chọn mô hình cho dự báo đúng *loại* giới hạn của từng họ mô hình, không chỉ thứ hạng của chúng.

Tài liệu tham khảo

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[2] R. A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.

[3] Q. Gu, Z. Li, J. Han. Generalized Fisher score for feature selection. *UAI*, 266–273, 2011.

[4] C. Spearman. The proof and measurement of association between two things. *American Journal of Psychology*, 15(1):72–101, 1904.

[5] I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016.

[6] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *JMLR*, 15:1929–1958, 2014.

[7] D. P. Kingma, J. Ba. Adam: a method for stochastic optimization. *ICLR*, 2015.

[8] F. Pedregosa và cộng sự. Scikit-learn: machine learning in Python. *JMLR*, 12:2825–2830, 2011.

[9] A. Paszke và cộng sự. PyTorch: an imperative style, high-performance deep learning library. *NeurIPS*, 2019.

A Phân công công việc, khai báo sử dụng AI và tái lập

Bảng 7: Phân công.

Họ tên	MSSV	Phần việc	Đóng góp (%)
Cả hai		EDA	
		Tiền xử lý dùng chung	
Nguyễn Hồng Linh	24280081	Naive Bayes (Multinomial, Gaussian)	100
		MLP	
		Báo cáo, slide	
Nguyễn Diệu Lê	24280080	Hồi quy softmax	100
		Random Forest; đối chiếu AdaBoost/Gradient Boosting	
		CNN; tổ hợp mô hình; notebook tái lập	
		Giao diện demo	

Khai báo sử dụng AI. Nhóm có dùng trợ lý lập trình AI (Claude Code, Anthropic). Phạm vi sử dụng: thiết kế tổng quan repository, các hàm kiểm tra (test), giao diện web demo, hình minh hoạ pipeline, thiết kế slide và khung trình bày báo cáo. Mọi mã nguồn, số liệu và kết luận đã được nhóm kiểm tra, chạy lại và chịu trách nhiệm.

Tái lập kết quả. Mã nguồn: https://github.com/dieule-0810/nhom10-mnist-classifiers.
Sau khi cài `requirements.txt`, notebook `notebooks/results.ipynb` nạp các mô hình cuối đã lưu và tính lại mọi con số của báo cáo trong vài phút (dữ liệu MNIST có sẵn trong `data/raw/`). Demo: `python -m demo.app`; kiểm tra bước chuẩn hoá ảnh vẽ: `python -m demo.robustness`. Chạy lại toàn bộ thí nghiệm mất khoảng 2 giờ trên CPU. Tái lập từng ô ma trận nhầm lẫn cần cùng seed và cùng số luồng BLAS (8).

B Giao diện demo

App Gradio cho người dùng vẽ một chữ số, chọn một trong sáu mô hình (năm mô hình đơn và CNN $\times$ 5, mục 5.7) và nhận 3 lớp có xác suất cao nhất (Hình 19). Vì ảnh vẽ khác ảnh MNIST ở cỡ, vị trí và độ dày nét, ảnh được đưa về quy ước chuẩn hoá của MNIST [1] (Thuật toán 5) rồi mới vào *chính* Pipeline đã lưu. Trên 300 ảnh test vẽ lại to gấp 7 lần và lệch tới ± 40 pixel, bước này giúp CNN giữ 99,3% dự đoán trên ảnh gốc, còn thu nhỏ thẳng cả khung làm accuracy của CNN rơi xuống 54,0%.

Thuật toán 5 Ảnh vẽ về dạng MNIST (`demo/drawing.py`)

Đầu vào: Ảnh vẽ RGBA 280×280 (nét tối trên nền trong suốt)

1: Đặt lên nền trắng, chuyển xám, đảo màu: mực $m \in [0, 1]$
2: Cắt theo khung nhỏ nhất chứa các pixel có $m > 0{,}05$
3: Thu nhỏ *giữ tỉ lệ* (Lanczos) sao cho cạnh dài bằng 20
4: Đặt vào khung 56×56, dịch sao cho tâm khối lượng $(\sum_{ij} i\, m_{ij} / \sum m,\ \sum_{ij} j\, m_{ij} / \sum m)$ về giữa; cắt lấy 28×28 ở giữa
5: Đặt $m \leftarrow 0$ ở các pixel có $m < 0{,}05$ (bỏ quầng mờ do đổi kích thước)
6: **trả về** vector 784 chiều cho `predict_proba`



Hình 19: Giao diện demo: CNN đoán đúng chữ số 4 vừa vẽ; bên phải là ảnh 28×28 mô hình thực sự nhận được.